

PPO Summary: From Policy Gradient Theorem to State-of-the-Art GAE Implementation

This document provides a concise summary of the 8-step mathematical journey from the foundational policy gradient theorem to the sophisticated Generalized Advantage Estimation (GAE) used in modern PPO implementations.

Step 1: Policy Gradient Foundation

Core Objective: Maximize expected return $J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[R(\tau)]$

Key Assumptions: - MDP Framework: States S , actions A , transitions $P(s'|s,a)$, rewards $R(s,a)$, discount γ - Differentiability: Policy $\pi_\theta(a|s)$ differentiable w.r.t. θ - Stationarity: Time-invariant dynamics and policy (momentarily during gradient computation)

Policy Gradient Theorem:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t) R_t \right]$$

Key Equations: - Return: $R_t = \sum_{k=t}^{T-1} \gamma^{k-t} r_k$ (causality principle) - Likelihood ratio trick: $\nabla_\theta \mathbb{E}_{x \sim p_\theta}[f(x)] = \mathbb{E}_{x \sim p_\theta}[f(x) \nabla_\theta \log p_\theta(x)]$

Rationale: Uses likelihood ratio trick and causality principle to derive unbiased gradient estimator. Log probability provides numerical stability, natural gradient interpretation, and mathematical elegance.

Step 2: The High Variance Problem and Baseline Subtraction

Problem: Basic policy gradient suffers from extremely high variance, making learning unstable.

Key Concepts: - **Variance:** Gradient estimates fluctuate around expected value across trajectory samples - **Bias:** Systematic error between estimator expectation and true gradient

Solution - Baseline Subtraction Theorem:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t) (R_t - b(s_t)) \right]$$

Key Insight: $\mathbb{E}_{a \sim \pi_\theta}[\nabla_\theta \log \pi_\theta(a|s) b(s)] = 0$

Optimal Baseline: $b^*(s) \approx V^\pi(s) = \mathbb{E}_{a \sim \pi_\theta}[Q^\pi(s, a)]$

Rationale: Any state-dependent baseline can be subtracted without introducing bias, with state value function being the variance-minimizing choice.

Step 3: Emergence of Advantage Functions

Mathematical Derivation: Using $\hat{V}(s)$ as baseline leads to advantage functions:

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$$

Advantage-Based Policy Gradient:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t | s_t) A^\pi(s_t, a_t) \right]$$

Critical Property: $\mathbb{E}_{a \sim \pi_\theta} [A^\pi(s, a)] = 0$ (zero-mean)

Mathematical Proof:

$$\mathbb{E}_{a \sim \pi_\theta} [A^\pi(s, a)] = \mathbb{E}_{a \sim \pi_\theta} [Q^\pi(s, a)] - V^\pi(s) = V^\pi(s) - V^\pi(s) = 0$$

Rationale: Advantages provide relative action quality information without introducing bias, ensuring unbiased policy gradient updates while reducing variance.

Step 4: Temporal Difference (TD) Estimation

Problem: Direct computation of $Q^\pi(s, a)$ and $V^\pi(s)$ is intractable.

TD Solution: Use Bellman equation to estimate advantages:

$$\delta_t^V = r_t + \gamma V(s_{t+1}) - V(s_t)$$

Key Assumptions: - Bellman Validity: $V^\pi(s_t) = \mathbb{E}_\pi [r_t + \gamma V^\pi(s_{t+1}) | s_t]$ - Markov Property: Future depends only on current state - Value Function Accuracy: $V(s) = \hat{V}(s)$

Bias-Variance Tradeoff: - **Monte Carlo Direction:** Low bias, high variance (using actual rewards) - **TD Direction:** High bias, low variance (using bootstrapped estimates)

Advantage Approximation: When $V(s) = \hat{V}(s)$, then $\delta_t^V = A^\pi(s_t, a_t)$

Rationale: TD errors provide computationally tractable advantage estimates, introducing the fundamental bias-variance tradeoff in advantage estimation.

Step 5: The Bias-Variance Tradeoff in Advantage Estimation

Core Dilemma: - **TD Error ($\gamma=0$):** $\hat{A}_t = \delta_t \rightarrow$ High bias, Low variance - **Monte Carlo ($\gamma=1$):** $\hat{A}_t = \sum_{k=0}^{\infty} \gamma^k \delta_{t+k} \rightarrow$ Low bias, High variance

High Variance Sources: 1. Stochastic rewards and environment transitions 2. Function approximation errors 3. Finite sampling

Bias Sources: 1. Value function approximation: $V_-(s) \neq V_+(s)$ 2. Bootstrapping with estimated values 3. Finite horizon truncation

Rationale: Understanding the sources of bias and variance enables principled design of advantage estimators that balance these competing objectives.

Step 6: Generalized Advantage Estimation (GAE) - The Optimal Solution

Mathematical Foundation:

$$\hat{A}_t^{GAE(\gamma, \lambda)} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}^V$$

Recursive Implementation:

$$\hat{A}_t^{GAE} = \delta_t + \gamma \lambda \hat{A}_{t+1}^{GAE}$$

Bias-Variance Control Parameter λ : - $\lambda = 0$: Pure TD $\rightarrow$ High bias, low variance - $\lambda = 1$: Pure MC $\rightarrow$ Low bias, high variance
- $\lambda \in (0,1)$: Optimal tradeoff $\rightarrow$ Exponentially weighted combination

Key Assumptions: 1. Exponential decay: Future information becomes less reliable exponentially 2. Geometric convergence: $|\lambda| < 1$ ensures series convergence 3. Finite horizon: Practical truncation at episode boundaries

Rationale: GAE provides principled interpolation between bias and variance, allowing practitioners to tune the λ parameter for optimal performance in specific domains.

Step 7: State-of-the-Art Implementation in PPO

Practical GAE Algorithm:

```
def gae_advantages(rewards, terminal_masks, values, discount, gae_param):
    advantages = []
    gae = 0.0
    for t in reversed(range(len(rewards))):
        delta = rewards[t] + discount * values[t + 1] * terminal_masks[t] - values[t]
        gae = delta + discount * gae_param * terminal_masks[t] * gae
        advantages.append(gae)
    return jnp.array(advantages[::-1])
```

Why Reversed Loop is Essential: - **Mathematical Dependency:** $\hat{A}_t^{GAE} = \delta_t + \gamma \lambda \hat{A}_{t+1}^{GAE}$ creates backward dependency chain - **Boundary Condition:** $\hat{A}_T^{GAE} = \delta_T$ (terminal condition) - **Computational Efficiency:** $O(T)$ linear time complexity - **Terminal State Handling:** Prevents advantage bleeding across episodes

Implementation Assumptions: - Neural network approximation: $V_{\theta}(s) \approx \hat{V}(s)$ - Backward processing for efficient computation - Terminal state handling: $V(s_{\text{terminal}}) = 0$ - Batch processing across parallel environments

Hyperparameter Sensitivity: - $[0.95, 0.99]$: Discount factor - $[0.9, 0.99]$: GAE parameter

Rationale: Efficient implementation that maintains mathematical correctness while enabling practical deployment in modern RL systems.

Step 8: Integration with PPO Loss Function

Final Policy Gradient with GAE:

$$\nabla_{\theta} J(\theta) \approx \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \hat{A}_t^{GAE} \right]$$

PPO Clipped Objective:

$$L^{CLIP}(\theta) = \mathbb{E}_t [\min(r_t(\theta) \hat{A}_t^{GAE}, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t^{GAE})]$$

where $r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)}$ is the importance sampling ratio.

Key Components:

1. **Importance Sampling Ratio:** Enables off-policy updates and sample reuse
 - Corrects for policy mismatch between data collection and updates

- Mathematical foundation:

$$\mathbb{E}_{\tau \sim \pi_{\theta}}[f(\tau)] = \mathbb{E}_{\tau \sim \pi_{\theta_{old}}} \left[\frac{\pi_{\theta}(\tau)}{\pi_{\theta_{old}}(\tau)} f(\tau) \right]$$

PPO's Hybrid On-Policy/Off-Policy Approach:

The use of importance sampling in PPO represents a **perfect example** of the on-policy/off-policy discussion in reinforcement learning, creating a sophisticated hybrid approach:

Traditional Distinction:

- **On-Policy Methods:** Learn about the policy being used to make decisions (e.g., REINFORCE, SARSA)
 - Theoretical guarantees
 - Stable training
 - Sample inefficient
- **Off-Policy Methods:** Can learn from data generated by different policies (e.g., Q-learning, DQN)
 - Sample efficient
 - Can learn from any data
 - Can be unstable
 - Distribution mismatch issues

PPO's Innovation: PPO bridges this gap through its two-phase process:

- **Experience Collection:** Use policy `__old` (on-policy data collection)
- **Multiple Updates:** Reuse same data for several gradient steps (off-policy learning with importance sampling correction)

The Fundamental Tradeoff PPO Solves: Traditional On-Policy: `__old` → collect data → one update → `__new` → discard data → repeat PPO Approach: `__old` → collect data → multiple updates → `__new` (reuse same data!)

Why This Matters:

- **Sample Efficiency:** Can reuse data multiple times (like off-policy methods)
- **Stability:** Clipping prevents destructive updates (like on-policy methods)
- **Practical Performance:** Works well across diverse domains
- **Theoretical Soundness:** Maintains convergence guarantees

This hybrid approach demonstrates that the on-policy/off-policy distinction isn't binary but rather a spectrum, with PPO finding an optimal balance between sample efficiency and training stability.

2. **Clipping Mechanism:** Prevents destructively large policy updates

$$\text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) = \begin{cases} 1 - \epsilon & \text{if } r_t(\theta) < 1 - \epsilon \\ r_t(\theta) & \text{if } 1 - \epsilon \leq r_t(\theta) \leq 1 + \epsilon \\ 1 + \epsilon & \text{if } r_t(\theta) > 1 + \epsilon \end{cases}$$

3. **Complete PPO Loss:**

$$L_{PPO}(\theta) = L^{CLIP}(\theta) - c_1 L^{VF}(\theta) + c_2 S[\pi_\theta]$$

Theoretical Properties: - Conservative updates through clipping - Monotonic improvement guarantees - Sample efficiency via experience reuse - Training stability through bounded policy changes

Rationale: Integration of GAE with PPO’s clipped objective creates a theoretically sound and practically effective algorithm that balances sample efficiency, training stability, and performance.

Summary of Key Benefits

1. **Variance Reduction:** GAE smooths noisy advantage estimates
2. **Bias Control:** parameter enables fine-tuning of bias-variance tradeoff
3. **Sample Efficiency:** Better gradient estimates accelerate learning
4. **Stability:** Reduced gradient noise and clipping improve training stability
5. **Credit Assignment:** Multi-horizon consideration improves temporal credit assignment

Theoretical Guarantees

- **Unbiasedness:** When $V(s) = \hat{V}(s)$, GAE provides unbiased advantage estimates
- **Consistency:** As $\gamma \rightarrow 1$, GAE approaches Monte Carlo estimates
- **Convergence:** Maintains policy gradient theorem guarantees while improving practical performance

This progression represents a principled evolution driven by the fundamental bias-variance tradeoff, culminating in state-of-the-art advantage estimation used in modern PPO implementations.